URL Shortener Web Application (Advance)

1. Introduction

In modern web usage, URLs are essential for accessing online resources. However, many URLs are very long and difficult to share or remember. A URL Shortener converts a long URL into a short and easy-to-use link.

This project focuses on building an Advanced URL Shortener Web Application that not only shortens URLs but also allows users to save and manage their shortened URLs using a login system. The application is developed using Flask as the backend framework, SQL Alchemy as the ORM for database management, and Bootstrap for frontend styling.

The main goal of this project is to design a secure, user-friendly, and functional web application that meets the objectives provided in the assignment.

2. Objectives of the Project

The objectives of this project are:

1. To shorten long URLs into unique and small URLs.
2. To store original and shortened URLs in a database.
3. To provide user authentication using signup and login functionality.
4. To validate URLs before shortening them.
5. To display previously shortened URLs on a history page.
6. To allow users to copy shortened URLs easily.
7. To redirect short URLs to the original URLs.

3. Technology Stack

Backend:

- Python (Flask Framework) – Handles routing and logic.
- Flask-Login – Manages user authentication and sessions.
- SQL Alchemy (ORM) – Used for database interaction.

- Validators library – Used to validate URLs.

Frontend:

- HTML – Structure of the web pages.
- CSS + Bootstrap – For responsive and attractive UI.

Database:

- SQLite – Lightweight relational database.

4. System Architecture

The system follows a client-server architecture:

1. The user interacts with the web pages (frontend).
2. Flask receives HTTP requests and processes them.
3. SQL Alchemy communicates with the database.
4. The system generates a short URL and stores it.
5. When a short URL is accessed, the user is redirected to the original URL.

The system contains the following pages:

- Signup Page
- Login Page
- Home Page (URL shortener)
- History Page

5. Database Design

Two tables are used:

User Table:

- id – Primary Key

- username – Unique
- password – Stored as hashed value

URL Table:

- id – Primary Key
- original_url – Stores the long URL
- short_code – Stores the generated short code
- user_id – Foreign key linked to User table

This relationship allows each user to store multiple URLs.

6. Project Workflow

1. User registers using the signup page.
2. Username is checked for uniqueness and valid length (5–9 characters).
3. Password is stored securely using hashing.
4. User logs in with valid credentials.
5. User enters a URL to shorten.
6. The system validates whether the URL is correct.
7. A unique short code is generated.
8. The URL and short code are saved in the database.
9. The short URL is displayed with a copy button.
10. User can view all previously shortened URLs on the history page.
11. When the short URL is accessed, the system redirects to the original URL.

7. URL Validation

The application validates user input URLs using the validators library.
If the URL is invalid, an error message is shown and the URL is not saved.

This ensures:

- Only valid URLs are stored.
- Database consistency is maintained.
- User mistakes are handled gracefully.

8. Security Implementation

Several security measures are implemented:

- Passwords are stored using hashing instead of plain text.
- Flask-Login manages sessions securely.
- Users must log in to access the URL shortener and history pages.
- Duplicate usernames are prevented.
- Each user can only view their own URLs.

9. Frontend Design

Bootstrap is used to design a responsive and user-friendly interface.

The UI contains:

- Input field for URL
- Shorten button
- Copy button
- Navigation bar
- History table

This improves user experience and usability.

10. Challenges Faced

1. Implementing login and signup functionality.
2. Preventing duplicate short URLs.
3. Handling database relationships between users and URLs.
4. Managing URL validation correctly.
5. Handling redirects using short codes.

11. Learning Outcomes

Through this project, the following concepts were learned:

- Flask routing and request handling
- Database modeling using SQL Alchemy
- User authentication using Flask-Login
- URL validation
- Secure password storage
- Bootstrap-based UI design
- Full-stack web development

12. Conclusion

The Advanced URL Shortener Web Application successfully meets all the objectives of the assignment. It allows users to shorten URLs, store them in a database, view their history, and securely access the system through login functionality. The application is efficient, user-friendly, and secure.

Future enhancements can include:

- Custom short URLs
- URL expiry feature
- Click analytics
- Admin panel